

STANDARD OPERATING PROCEDURE

Using DAX Filter Functions in Power BI / Analysis Services

A step-by-step reference guide for students on applying, combining, and troubleshooting DAX filter and filter-modifier functions — with syntax, worked examples, and common mistakes for every function.

AUDIENCE	Students learning DAX / Power BI data modeling
PRIMARY REFERENCE	Microsoft Learn – Filter functions (DAX)
REFERENCE URL	learn.microsoft.com/en-us/dax/filter-functions-dax
APPLIES TO	Power BI Desktop, Excel Power Pivot, Analysis Services,
Fabric	
VERSION / DATE	v1.0 — 30 August 2026

Prepared as a training / classroom reference. Practice examples use the public AdventureWorks DW sample model referenced in Microsoft's official documentation.

Table of Contents

1. Purpose & Scope	3
2. Prerequisites	4
3. Core Concepts Refresher	5
4. Standard Procedure: How to Approach Any Filtering Problem	6
5. Quick Decision Guide	7
6. Detailed Function Reference	8
6.1 FILTER	8
6.2 ALL	9
6.3 ALLEXCEPT	10
6.4 ALLSELECTED	11
6.5 ALLNOBLANKROW	13
6.6 ALLCROSSFILTERED	14
6.7 CALCULATE	15
6.8 CALCULATETABLE	16
6.9 REMOVEFILTERS	17
6.10 KEEPFILTERS	18
6.11 EARLIER	19
6.12 EARLIEST	20
6.13 LOOKUPVALUE	21
6.14 SELECTEDVALUE	23
7. Cross-Cutting Best Practices & Pitfalls	25
8. Adjacent Functions Outside This SOP's Scope	26
9. Practice Exercises for Students	27
10. Quick Reference Summary Table	28
11. References	29

1. Purpose & Scope

Purpose. This Standard Operating Procedure (SOP) gives students a repeatable, step-by-step method for choosing, writing, and troubleshooting **DAX filter functions** — the group of functions that let you read and modify the *filter context* of a calculation in Power BI, Excel Power Pivot, Analysis Services Tabular, and Microsoft Fabric semantic models.

Scope. This SOP covers every function Microsoft classifies under the official *Filter functions (DAX)* category that is used for classic filter-context manipulation:

Function	What it does
FILTER	Returns a filtered subset of a table as a table.
ALL	Removes filters from a table or specific column(s).
ALLEXCEPT	Removes filters from a table except on named columns.
ALLSELECTED	Removes filters introduced inside the query while keeping outer/report filters.
ALLNOBLANKROW	Like ALL, but excludes the special relationship "blank row".
ALLCROSSFILTERED	Clears cross-filters propagated onto a table (e.g. via many-to-many).
CALCULATE	Evaluates a scalar expression in a modified filter context.
CALCULATETABLE	Evaluates a table expression in a modified filter context.
REMOVEFILTERS	Explicitly clears filters (modern, readable alternative to ALL as a modifier).
KEEPFILTERS	Adds a filter without overwriting existing filters on the same column.
EARLIER / EARLIEST	Access an outer row-context value inside a nested row-context loop.
LOOKUPVALUE	Looks up a value using one or more search conditions, independent of relationships.
SELECTEDVALUE	Returns the single selected value of a column, or a fallback.

OUT OF SCOPE

This SOP does not cover the newer *visual-calculation-only* and window functions that Microsoft also lists on the same reference page (**INDEX** , **WINDOW** , **OFFSET** , **RANK** , **FIRST** , **LAST** , **NEXT** , **PREVIOUS** , **MOVINGAVERAGE** , **RUNNINGSUM** , **ROWNUMBER** , **ORDERBY** , **PARTITIONBY** , **MATCHBY** , **LOOKUP** , **LOOKUPWITHTOTALS**). These apply only inside Power BI's Visual Calculations feature and follow a different mental model (matrix axis position rather than filter context). They are listed in Section 8 for completeness only.

Primary reference used for this SOP: Microsoft Learn, *Filter functions (DAX)* — <https://learn.microsoft.com/en-us/dax/filter-functions-dax> , plus each function's individual reference page on the same site.

2. Prerequisites

Before working through this SOP, a student should have:

1. Access to **Power BI Desktop** (free) or **Excel with Power Pivot**, and a working data model with at least one fact table and one or more dimension tables joined by relationships.
2. Basic familiarity with writing a simple measure, e.g. `Total Sales = SUM ( Sales[Sales Amount] )`.
3. A sample model to practice on. Microsoft's official examples in this SOP use the public **AdventureWorks DW** sample model, downloadable from <https://aka.ms/dax-docs-samples>. Any star-schema model with a Sales/Orders fact table and Date, Product, and Customer dimensions will work just as well.
4. Understanding of the difference between a **column** reference (`Table[Column]`) and a **measure** reference (`[Measure]`), since several filter functions restrict which one you may use as an argument.

3. Core Concepts Refresher

Every filter function in this SOP exists to manipulate one thing: the **filter context**. Read this section before Section 5 — skipping it is the single biggest cause of student confusion with DAX.

3.1 FILTER CONTEXT

The set of filters (from slicers, rows/columns of a visual, or explicit DAX filter arguments) that is applied before a measure is evaluated. Filter functions read, add to, replace, or clear this context.

3.2 ROW CONTEXT

The "current row" that DAX is evaluating — automatic inside a calculated column, and manufactured by iterator functions such as **FILTER** , **SUMX** , **EARLIER** 's host loop, etc. Row context does *not* automatically filter other calculations — it must be converted.

3.3 CONTEXT TRANSITION

The process by which **CALCULATE** (or **CALCULATETABLE**) turns the current row context into an equivalent filter context, one column at a time, for every column in the table. This is why wrapping an expression in **CALCULATE** inside a calculated column can change its result even with no explicit filter arguments.

3.4 WHY THIS MATTERS FOR FILTER FUNCTIONS

Almost every function in Section 5 does exactly one of three things to the filter context:

Action	Functions
Remove existing filters (fully or partially)	ALL, ALLEXCEPT, ALLNOBLANKROW, ALLSELECTED, ALLCROSSFILTERED, REMOVEFILTERS
Add / replace filters	CALCULATE, CALCULATETABLE, FILTER (as an argument), KEEPFILTERS (adds without replacing)
Read a value out of the current context	SELECTEDVALUE, LOOKUPVALUE, EARLIER, EARLIEST

4. Standard Procedure: How to Approach Any Filtering Problem

Follow this six-step procedure every time a calculation needs a filter function. It is the workflow experienced DAX authors use, made explicit for students.

1. **Restate the requirement in plain English** before writing any DAX. Example: "I need total sales for the selected product category, but always across *all* years, regardless of the year slicer."
2. **Identify the current filter context** the visual/measure will run in (which slicers, rows, columns, or explicit filters apply).
3. **Identify exactly which columns/tables need their filters changed**, and whether you need to remove, add, or keep filters on them. Use the **Quick Decision Guide** in Section 5 to pick the right function(s).
4. **Wrap the target expression in** `CALCULATE` or `CALCULATETABLE` and pass the chosen filter function(s) as arguments. A scalar result (a number) needs `CALCULATE` ; a table result needs `CALCULATETABLE` .
5. **Test in a matrix visual** with the relevant fields on rows/columns and slicers applied, and compare the result against a hand-calculated expectation for at least two different filter combinations.
6. **Document the measure**: add a one-line description (Power BI > Measure tools > Description) explaining which filters are being removed/added and why, so the next person (or your future self) does not have to reverse-engineer it.

5. Quick Decision Guide

Use this table as a lookup during Step 3 of the procedure above.

If your goal is to...	...use this function
Wipe out every filter in the entire model	<code>ALL()</code> or <code>REMOVEFILTERS()</code>
Remove filters from one whole table, keep filters on other tables	<code>ALL(Table)</code> / <code>REMOVEFILTERS(Table)</code>
Remove filters from specific column(s) only, keep the rest	<code>ALL(Col1, Col2...)</code> / <code>REMOVEFILTERS(Col1, Col2...)</code>
Remove filters from every column in a table <i>except</i> one or two	<code>ALLEXCEPT(Table, Col1, ...)</code>
Compute a grand/visual total that must ignore the row/column slicer on a matrix but respect page/report-level filters	<code>ALLSELECTED()</code>
Count/sum rows of a table without letting the "unknown member" blank row distort results	<code>ALLNOBLANKROW</code>
Ignore a many-to-many cross-filter applied to a table	<code>ALLCROSSFILTERED(Table)</code>
Get a scalar number evaluated under a different/extra filter	<code>CALCULATE(<scalar expr>, <filters>)</code>
Get a filtered <i>table</i> (for use inside SUMX, COUNTROWS, etc.) under a different filter	<code>CALCULATETABLE(<table expr>, <filters>)</code>
Apply a complex row-by-row condition that a simple <code>Column = value</code> filter can't express	<code>FILTER(Table, <boolean expr>)</code> passed into <code>CALCULATE/CALCULATETABLE</code>
Add a narrowing filter but keep (intersect with) whatever filter is already applied to that column	<code>KEEPFILTERS(<filter>)</code> inside <code>CALCULATE</code>
Reference the outer row's column value while iterating in a calculated column (ranking, running totals)	<code>EARLIER(Column)</code> or <code>EARLIEST(Column)</code>
Pull a value from another table using conditions that don't match an existing relationship	<code>LOOKUPVALUE(...)</code>
Show a value only when exactly one item is selected/filtered, else a message or blank	<code>SELECTEDVALUE(Column, <fallback>)</code>

6. Detailed Function Reference

Each function below follows the same structure: Purpose, Syntax, Parameters, Step-by-Step Usage, a Worked Example (traceable to Microsoft's official documentation), and Common Student Mistakes.

6.1 FILTER

Returns: TABLE

Category: Base filter function

Used inside: CALCULATE / CALCULATETABLE / iterators

Purpose: Returns a table containing only the rows of another table (or table expression) that satisfy a Boolean condition. FILTER is rarely used alone — it is almost always passed as an argument into CALCULATE, CALCULATETABLE, or an iterator like SUMX/COUNTROWS.

SYNTAX

```
FILTER ( <table>, <filter> )
```

Parameter	Definition
table	The table to filter. Can be a base table or any expression that returns a table.
filter	A Boolean expression evaluated row-by-row, e.g. <code>[Amount] > 0</code> or <code>[Region] = "France"</code> .

STEP-BY-STEP USAGE

1. Identify the table you need a subset of.
2. Write the row-level Boolean condition (it can reference any column visible in that row context, including columns from related tables via RELATED).
3. Wrap `FILTER(table, condition)` as an argument to CALCULATE/CALCULATETABLE, or pass it straight into an iterator such as SUMX or COUNTROWS.
4. Test with COUNTROWS(FILTER(...)) first to sanity-check how many rows survive the filter before using it inside a larger calculation.

WORKED EXAMPLE

Scenario: report Internet sales excluding the United States.

```
NON USA Internet Sales =
SUMX (
    FILTER (
        InternetSales_USD,
        RELATED ( SalesTerritory[SalesTerritoryCountry] ) <> "United States"
    ),
    InternetSales_USD[SalesAmount_USD]
)
```


Result: Total Internet sales of \$26,054,827.45 becomes \$16,665,347.67 once United States rows (\$9,389,479.79) are excluded — verified by slicing on `SalesTerritoryCountry` in a PivotTable and comparing totals.

COMMON STUDENT MISTAKES

- Using `FILTER` directly as the *only* filter argument to `CALCULATE` when a simple Boolean filter (`Table[Column] = value`) would do — this is slower. Official best-practice guidance: *avoid using FILTER as a filter argument* when a plain Boolean expression is possible; reserve `FILTER` for conditions a Boolean expression cannot express (e.g. referencing a related table, or an aggregation-based condition).
- Forgetting that `FILTER` returns a **table**, not a scalar — trying to use `= FILTER(Sales, ...)` directly as a measure value will error; it must be consumed by `SUMX`, `COUNTROWS`, `CALCULATE`, etc.
- Writing conditions against a measure instead of a column inside `FILTER` used as a `CALCULATE` Boolean argument (measures are allowed inside `FILTER`'s table-scan, but not inside a plain Boolean filter argument to `CALCULATE`).

6.2 ALL

Returns: `TABLE / COLUMN`

Category: Filter modifier

Used inside: `CALCULATE / CALCULATETABLE`

Purpose: Returns all rows of a table, or all values of a column, ignoring any filters currently applied. Used to clear filters so a calculation can produce a grand total, a ratio-to-total, or any "regardless of what's sliced" result.

SYNTAX

```
ALL ( [ <table> | <column> [, <column> [, <column> [...]]] ] )
```

Parameter	Definition
table	The table to clear all filters from. Must be a base table reference (not a table expression).
column	One or more columns, all from the same table, to clear filters from individually.

THREE WAYS TO CALL ALL

Form	Effect
<code>ALL ()</code>	Removes all filters everywhere (modifier-only, cannot be used as a table expression).
<code>ALL (Table)</code>	Removes all filters from the specified table only.
<code>ALL (Col1, Col2...)</code>	Removes filters only from the named columns (all from one table); other filters on that table remain.

STEP-BY-STEP USAGE

1. Decide the scope: do you need to clear the whole table, or just specific columns? (Use the table above.)

2. Place ALL(...) as a filter argument inside CALCULATE or CALCULATETABLE — it is a filter *modifier*, not a standalone calculation.
3. Combine with an aggregation (SUM, SUMX, COUNTROWS) as the CALCULATE expression to build the "denominator" of a ratio.
4. Watch for **auto-exist**: when two or more columns from the same table are filtered together (e.g. by slicers) and a measure uses ALL() on that table, DAX may only recompute over the combinations of values that already co-exist in the data, not literally "everything." Test with multiple simultaneous slicers to confirm your ALL() measure behaves as expected.

WORKED EXAMPLE

Scenario: show each product category's share of total reseller sales, regardless of which category is currently displayed in a row.

```
All Reseller Sales Ratio =
DIVIDE (
    SUMX ( ResellerSales_USD, ResellerSales_USD[SalesAmount_USD] ),
    SUMX ( ALL ( ResellerSales_USD ), ResellerSales_USD[SalesAmount_USD] )
)
```

Result: In a matrix with CalendarYear on rows and ProductCategoryName on columns, each cell shows its share of the overall grand total (e.g. Bikes 2007 = 31.71%), and every cell's denominator stays fixed at the full \$109.8M total.

COMMON STUDENT MISTAKES

- Calling `ALL()` with zero arguments expecting it to return a table — the no-argument form is a pure modifier and cannot be used where a table object is required (e.g., cannot pass to SUMX directly).
- Passing a table *expression* (like the output of FILTER) into ALL — ALL only accepts base table or base column references.
- Expecting ALL(Column) to also clear filters on sibling columns of the same table — it only clears the named column(s); use ALLEXCEPT or ALL(Table) if the whole table needs clearing.

6.3 ALLEXCEPT

Returns: TABLE

Category: Filter modifier

Used inside: CALCULATE / CALCULATETABLE

Purpose: Removes all context filters from a table *except* the filters applied to the column(s) you name. It is a shortcut for "clear everything on this table except these one or two columns," which would otherwise require listing every other column inside ALL().

SYNTAX

```
ALLEXCEPT ( <table>, <column> [, <column> [, ...]] )
```

Parameter	Definition
table	Base table to clear filters from. Required, must be a base table.
column	One or more base columns (from the same table) whose filters must be <i>preserved</i> .

STEP-BY-STEP USAGE

1. Identify the table with many filterable columns (e.g. a Date table with Year, Quarter, Month, Weekday...).
2. Identify the one or two columns whose current filter must survive (e.g. Year).
3. Write `CALCULATE(<expr>, ALLEXCEPT(Table, Table[ColumnToKeep]))`.
4. Test by slicing on a column you did *not* list — the measure should ignore that slicer — then slice on the column you did list — the measure should respect that slicer.

WORKED EXAMPLE

Scenario: sum of sales that ignores every Date-table slicer except CalendarYear.

```
Sales By Year Only =
CALCULATE (
    SUM ( ResellerSales_USD[SalesAmount_USD] ),
    ALLEXCEPT ( DateTime, DateTime[CalendarYear] )
)
```

Result: Slicing the report by Month or Weekday has no effect on this measure — it always returns the full-year total — but slicing by CalendarYear still changes the result.

COMMON STUDENT MISTAKES

- Passing columns from a *different* table than the first argument — all column arguments must belong to the table named as the first argument.
- Using ALLEXCEPT when only one or two columns need clearing (the reverse case) — that's a job for `ALL(Col1, Col2)` instead, which is easier to read.
- Forgetting that ALLEXCEPT still leaves row-level relationships from *other* tables filtering the target table (e.g. if Product filters Sales via a relationship, that still applies) — ALLEXCEPT only clears filters that were directly on the named table's own columns.

6.4 ALLSELECTED

Returns: TABLE / COLUMN

Category: Filter modifier

Used inside: CALCULATE / CALCULATETABLE

Purpose: Removes filters that were introduced *inside* the current visual/query (typically row and column groupings of a matrix), while keeping filters that come from *outside* it — slicers, report-level filters, or filters

explicitly passed higher up. This is the standard way to build "visual totals" that respect page filters but ignore the matrix's own row/column breakdown.

SYNTAX

```
ALLSELECTED ( [ <tableName> | <columnName> [, <columnName> [,...]] ] )
```

Parameter	Definition
tableName	Optional. Name of an existing table.
columnName	Optional. One or more columns from the same table.

STEP-BY-STEP USAGE

1. Build the matrix/table visual with the groupings that need a "visual total" (e.g. Year on rows, Product Category on columns).
2. Apply any slicers or page-level filters you want the total to *respect* (e.g. Sales Territory Group = Europe).
3. Write `CALCULATE(<expr>, ALLSELECTED())` for a full visual grand total, or `ALLSELECTED(Table[Column])` to total across just one axis while keeping the other axis's breakdown.
4. Compare against `CALCULATE(<expr>, ALL(Table))` in the same report — ALL ignores the slicers too, ALLSELECTED does not; this contrast is the fastest way for a student to see the difference.

WORKED EXAMPLE

Scenario: a matrix filtered to Sales Territory Group = Europe and Promotion Type = Volume Discount needs subtotal rows for "all years" and "all categories," plus a grand total, that still honor those two filters.

```
Reseller Sales Amount = SUM ( 'Reseller Sales'[Sales Amount] )

Reseller Visual Total =
CALCULATE ( SUM ( 'Reseller Sales'[Sales Amount] ), ALLSELECTED() )

Reseller Visual Total for All of Calendar Year =
CALCULATE ( SUM ( 'Reseller Sales'[Sales Amount] ), ALLSELECTED ( 'Date'[Calendar Year] ) )

Reseller Grand Total =
CALCULATE ( SUM ( 'Reseller Sales'[Sales Amount] ), ALL ( 'Reseller Sales' ) )
```

Result: `Reseller Visual Total` returns \$877,006.80 (Europe + Volume Discount only), while `Reseller Grand Total` returns \$80,450,596.98 (every territory and promotion) — proving ALLSELECTED kept the two outer filters but ALL discarded them.

COMMON STUDENT MISTAKES

- Assuming ALLSELECTED() behaves exactly like ALL() — they diverge as soon as any slicer or report filter is applied; test both side by side, never assume.
- Using ALLSELECTED inside a calculated column, where the concept of "the current visual's selection" is not well defined — it is designed for measures evaluated inside a report/query, not row-by-row column calculations.
- Forgetting that explicit filters written elsewhere in the same CALCULATE call still apply on top of ALLSELECTED's result.

6.5 ALLNOBLANKROW

Returns: TABLE / COLUMN

Category: Filter modifier

Used inside: CALCULATE / COUNTROWS

Purpose: Behaves like ALL, but on the "one" (parent) side of a relationship it also excludes the automatically generated blank/"Unknown member" row that Power BI creates when child-table rows don't match any parent row. Use it whenever ALL is producing an unexpected extra blank category or an off-by-one row count.

SYNTAX

```
ALLNOBLANKROW ( { <table> | <column> [, <column> [, <column> [,...]] ] } )
```

Parameter	Definition
table / column	Exactly one table, or one or more columns, to clear filters from while ignoring the special blank row.

STEP-BY-STEP USAGE

1. Notice the symptom: a matrix on the parent side of a relationship shows a mystery blank row/label at the bottom, or a ROWCOUNT is exactly one higher than expected.
2. Confirm the cause: a child table has rows with a key that doesn't match any parent row, or a null relationship key.
3. Replace `ALL(Table)` with `ALLNOBLANKROW(Table)` in the affected measure.
4. Re-test the row count / total — the artificial blank row should disappear from the result while genuine data is unaffected.

WORKED EXAMPLE

```
Countrows ALLNOBLANKROW of datetime = COUNTROWS ( ALLNOBLANKROW ( 'DateTime' ) )
Countrows ALL of datetime           = COUNTROWS ( ALL ( 'DateTime' ) )
```

Result: ALL returns 1,281 rows (includes the Unknown-member blank row created because some ResellerSales rows don't match any DateTime row); ALLNOBLANKROW returns 1,280 — the true row count of the DateTime table.

COMMON STUDENT MISTAKES

- Using ALLNOBLANKROW on the *child* table expecting it to remove genuinely blank rows in that table — it only removes the special automatic blank row that appears on the **parent** side of a relationship, not ordinary null values.
- Reaching for ALLNOBLANKROW as a first choice instead of ALL — use plain ALL by default; switch to ALLNOBLANKROW only when the blank-row artifact is actually causing a problem.

6.6 ALLCROSSFILTERED

Returns: N/A (modifier only)

Category: Filter modifier

Used inside: CALCULATE

Purpose: Clears all filters — including those arriving indirectly through a many-to-many or bidirectional cross-filter relationship — that are currently applied to a table. Plain ALL(Table) does not always reach cross-filters propagated this way; ALLCROSSFILTERED does.

SYNTAX

```
ALLCROSSFILTERED ( <table> )
```

Parameter	Definition
table	The table to clear all direct and cross-filters from.

STEP-BY-STEP USAGE

1. Confirm there is a many-to-many (or bidirectional) relationship reaching the fact table you're measuring.
2. Write a baseline measure with `CALCULATE(<expr>, ALL(FactTable))` and observe it is still being filtered by the many-to-many dimension.
3. Replace with `CALCULATE(<expr>, ALLCROSSFILTERED(FactTable))` to fully clear it.

WORKED EXAMPLE

```
TotalQuantity1 = CALCULATE ( SUM ( FactInternetSales[OrderQuantity] ), ALLCROSSFILTERED
( FactInternetSales ) )
TotalQuantity2 = CALCULATE ( SUM ( FactInternetSales[OrderQuantity] ), ALL
( FactInternetSales ) )
```

Result: With a many-to-many relationship between FactInternetSales and DimSalesReason, slicing by Sales Reason Name still changes `TotalQuantity2` (ALL), but `TotalQuantity1` (ALLCROSSFILTERED) stays fixed at 60,398 for every slicer value.

COMMON STUDENT MISTAKES

- Reaching for this function when there is no many-to-many/bidirectional relationship in play — in a normal single-direction star schema, plain ALL already does the job.
- Forgetting ALLCROSSFILTERED, like REMOVEFILTERS and ALLCROSSFILTERED's siblings, is a modifier only — it cannot be used where a table object is expected.

6.7 CALCULATE

Returns: SCALAR

Category: Core evaluation function

The single most important DAX function

Purpose: Evaluates an expression (typically an aggregation) in a filter context that CALCULATE itself modifies using the filter arguments you supply. Almost every other filter function in this SOP exists to be used *inside* a CALCULATE call.

SYNTAX

```
CALCULATE ( <expression> [, <filter1> [, <filter2> [,...]]] )
```

Parameter	Definition
expression	The scalar expression to evaluate — behaves like a measure.
filter1, filter2...	Optional Boolean filter expressions, table filter expressions, or filter-modifier functions (ALL, ALLEXCEPT, REMOVEFILTERS, KEEPFILTERS, USERELATIONSHIP, CROSSFILTER). Multiple filters combine with AND.

STEP-BY-STEP USAGE

1. Write the base aggregation you want to compute (e.g. `SUM(Sales[Sales Amount])`).
2. Decide what filter change is required: add a new condition, replace an existing one, or remove one — pick the matching filter argument type (Boolean expression, FILTER() table expression, or a modifier function).
3. Wrap: `CALCULATE(<base expr>, <filter args>)` .
4. Remember the default overwrite rule: if a filter argument targets a column that already has a filter in the surrounding context, CALCULATE's filter *replaces* it (unless wrapped in KEEPFILTERS); if the column had no filter yet, CALCULATE's filter is *added*.
5. If used inside a calculated column or an iterator's row context, remember CALCULATE triggers **context transition** — the current row's values become filters — even with no explicit filter arguments.

WORKED EXAMPLE

```
Blue Revenue =
CALCULATE ( SUM ( Sales[Sales Amount] ), 'Product'[Color] = "Blue" )

Revenue % Total Channel =
DIVIDE (
    SUM ( Sales[Sales Amount] ),
    CALCULATE ( SUM ( Sales[Sales Amount] ), REMOVEFILTERS ( 'Sales Order'[Channel] ) )
)
```

Result: Total sales of \$109,809,274.20 filters down to \$9,602,850.97 for Blue Revenue; the channel-ratio measure returns 26.74% for Internet and 73.26% for Reseller, each divided by the channel-independent grand total.

COMMON STUDENT MISTAKES

- Believing a CALCULATE filter *adds on top of* an existing filter on the same column by default — it actually **overwrites** it, unless you wrap the filter in KEEPFILTERS.
- Writing a Boolean filter that references a measure, uses a nested CALCULATE, or scans a table outside an aggregation — all disallowed inside a plain Boolean filter argument.
- Not realizing CALCULATE with *no* filter arguments still does something meaningful inside row context (context transition) — a common source of "why did adding CALCULATE() with nothing inside change my calculated column result?" confusion.

6.8 CALCULATETABLE

Returns: TABLE

Category: Core evaluation function

CALCULATE's table-returning twin

Purpose: Identical mechanics to CALCULATE, but the first argument is a table expression instead of a scalar, and the function returns a table. Use it whenever you need a filtered/derived table for further processing (as a calculated table, or fed into SUMX/COUNTROWS/another function).

SYNTAX

```
CALCULATETABLE ( <expression> [, <filter1> [, <filter2> [,...]] ] )
```

Parameter	Definition
expression	A model table, or any expression that returns a table.
filter1, filter2...	Same filter argument types as CALCULATE; multiple filters combine with AND.

STEP-BY-STEP USAGE

1. Identify the base table you want a modified-context version of.

2. Apply your filter argument(s) exactly as you would with CALCULATE.
3. Feed the resulting table into an aggregator (SUMX, COUNTROWS, AVERAGEX) or use it directly as a calculated table.

WORKED EXAMPLE

```
Internet Sales to 2006 ratio =
SUMX (
    CALCULATETABLE ( InternetSales_USD, 'DateTime'[CalendarYear] = 2006 ),
    [SalesAmount_USD]
)
```

Result: Regardless of which year is selected in a report row, this expression always sums 2006 Internet sales (\$5,681,440.58), letting a "ratio to 2006" column be built per row.

COMMON STUDENT MISTAKES

- Using CALCULATE when the first argument is actually a table — DAX enforces this, but students often first hit "a table of multiple values was supplied where a single value was expected" and don't realize the fix is switching to CALCULATETABLE.
- Forgetting boolean filters inside CALCULATETABLE can only reference a single column (no OR-across-columns tricks) unless combined via FILTER or the || operator on the same column.

6.9 REMOVEFILTERS

Returns: N/A (modifier only)

Category: Filter modifier

Modern replacement for ALL as a modifier

Purpose: Explicitly clears filters from the specified table(s)/column(s), or from everything if called with no arguments. It exists to make CALCULATE/CALCULATETABLE code more readable — where earlier DAX authors used `ALL()` purely as a modifier, REMOVEFILTERS states the intent directly and cannot be confused with "return a table."

SYNTAX

```
REMOVEFILTERS ( [ <table> | <column> [, <column> [, <column> [,...]]] ] )
```

Parameter	Definition
<code>table / column</code>	Optional. What to clear filters from; omit both for "clear everything."

STEP-BY-STEP USAGE

1. Anywhere you would historically write `ALL()`, `ALL(Table)`, or `ALL(Column)` purely to clear a filter inside CALCULATE/CALCULATETABLE (not to return a table object elsewhere), prefer REMOVEFILTERS for clarity.

2. Use the table/column argument to scope exactly what gets cleared, same rules as ALL.
3. If your tool/engine version doesn't support REMOVEFILTERS (older Excel Power Pivot), fall back to ALL — behavior for modifier use is equivalent.

WORKED EXAMPLE

```
%Sales = DIVIDE ( [TotalSales], CALCULATE ( [TotalSales], REMOVEFILTERS() ) )
```

Result: Each product category's `%Sales` divides its own total by the grand total with every filter removed — Bikes returns 96.46%, Accessories 2.39%, Clothing 1.16% of the \$29.36M internet sales total shown in the example.

COMMON STUDENT MISTAKES

- Trying to use REMOVEFILTERS to *return a table object* (e.g., passing it to SUMX directly) — like ALLCROSSFILTERED, it is modifier-only and has no independent table-returning form.
- Not realizing REMOVEFILTERS(Table) and ALL(Table) are functionally the same when used as a CALCULATE modifier — the choice between them is a readability/style decision, not a behavioral one.

6.10 KEEPFILTERS

Returns: value from wrapped expression

Category: Filter modifier

Used inside: CALCULATE / CALCULATETABLE

Purpose: Changes CALCULATE's default "overwrite" behavior into "intersect." Normally, a CALCULATE filter argument replaces any existing filter on the same column; wrapping that argument in KEEPFILTERS makes it *add to* (narrow) the existing filter instead.

SYNTAX

```
KEEPFILTERS ( <expression> )
```

Parameter	Definition
expression	The filter expression (Boolean or table) whose interaction with existing filters should be an intersection, not a replacement.

STEP-BY-STEP USAGE

1. Notice the symptom that calls for KEEPFILTERS: a nested/outer filter on a column is being silently discarded because an inner CALCULATE also filters that same column.
2. Wrap the inner CALCULATE's filter argument in `KEEPFILTERS(...)`.
3. Re-test: the result should now reflect the *intersection* of the outer and inner filters on that column, not just the inner one.

WORKED EXAMPLE

```

$$ in WA and OR, filtered further to WA and BC (no KEEPFILTERS) =
CALCULATE (
    CALCULATE ( [Internet Total Sales], 'Geography'[State Province Code] = "WA" ||
'Geography'[State Province Code] = "OR" ),
    'Geography'[State Province Code] = "WA" || 'Geography'[State Province Code] = "BC"
) -- returns $3,638,239.88 (WA+OR wins; outer WA+BC filter is overwritten)

$$ using KEEPFILTERS =
CALCULATE (
    CALCULATE ( [Internet Total Sales], KEEPFILTERS ( 'Geography'[State Province Code]
= "WA" || 'Geography'[State Province Code] = "OR" ) ),
    'Geography'[State Province Code] = "WA" || 'Geography'[State Province Code] = "BC"
) -- returns $2,467,248.34 (intersection = WA only)

```

Result: Without KEEPFILTERS the inner filter (WA, OR) simply overwrites the outer one (WA, BC). With KEEPFILTERS, DAX intersects the two sets on the same column, leaving only "WA" — matching the plain "\$\$ in WA" baseline exactly.

COMMON STUDENT MISTAKES

- Expecting KEEPFILTERS to affect columns *other* than the one named in its own expression — it only changes overwrite-vs-intersect behavior for the specific column(s) referenced inside it.
- Using KEEPFILTERS when the real goal is simply "don't remove any filters" — if there is no competing CALCULATE filter on the same column, KEEPFILTERS has no visible effect at all, which often confuses students testing it in isolation.

6.11 EARLIER

Returns: value

Category: Row-context function

Mainly used in: calculated columns

Purpose: Inside a nested row-context loop (for example, a FILTER running inside a calculated column), EARLIER lets you reach back and read the value of a column from the *outer* row context, rather than the innermost one. This enables row-by-row comparisons across an entire table, such as ranking.

SYNTAX

```
EARLIER ( <column>, <number> )
```

Parameter	Definition
column	A column, or expression resolving to a column, to read the outer value of.
number	Optional, defaults to 1. How many row-context levels to step outward (2 = two levels out, etc.).

STEP-BY-STEP USAGE

1. Confirm you are inside a **calculated column** (EARLIER needs a pre-existing outer row context; it errors without one).
2. Identify the outer value you need to "carry into" a nested table scan (e.g., FILTER).
3. Call `EARLIER(Table[Column])` inside that nested scan to compare each inner row against the outer row's value.
4. Wrap the FILTER + EARLIER combination in COUNTROWS (or another aggregator) to produce the final calculated-column value.
5. Be mindful of performance on large tables — EARLIER can force roughly row² comparisons; avoid it on very large fact tables and prefer RANKX or window functions where available.

WORKED EXAMPLE

Scenario: rank each product subcategory by total sales, highest first, using only calculated-column logic.

```
SubCategorySalesRanking =
COUNTROWS (
    FILTER (
        ProductSubcategory,
        EARLIER ( ProductSubcategory[TotalSubcategorySales] ) <
        ProductSubcategory[TotalSubcategorySales]
    )
) + 1
```

Result: Road Bikes (\$40.55M) ranks 1, Mountain Bikes (\$34.31M) ranks 2, down to Lights/Panniers (blank sales) ranking 36 — each row counts how many other rows outsell it, then adds 1 so the top seller isn't blank/zero.

COMMON STUDENT MISTAKES

- Calling EARLIER inside a **measure** instead of a calculated column — there is no outer row context in a measure evaluated from a visual, so EARLIER returns an error there.
- Using EARLIER when the modern, faster, and more readable `RANKX` function (or a visual-calculation RANK) would solve the same ranking problem — EARLIER is mainly a teaching tool today for understanding nested row context; prefer RANKX in production models.
- Nesting three or more row contexts and losing track of which "outer" level EARLIER(..., 2) refers to — sketch the loops on paper before writing the formula.

6.12 EARLIEST

Returns: value

Category: Row-context function

EARLIER's "go to the outermost level" sibling

Purpose: Returns the value of a column at the *outermost* row-context evaluation pass currently active — functionally equivalent to `EARLIER(column, <deepest available level>)`. Used far less often than EARLIER; mainly relevant with three or more nested row contexts.

SYNTAX

```
EARLIEST ( <column> )
```

Parameter	Definition
<code>column</code>	A reference to the column whose outermost row-context value you want.

STEP-BY-STEP USAGE

1. Only reach for EARLIEST once you have **two or more** nested row-context loops and specifically need the very first (outermost) one, not an intermediate level.
2. If you only have a single level of nesting, EARLIER(column) and EARLIEST(column) return the same result — use EARLIER by default since it is more common and self-documenting with its optional level argument.

WORKED EXAMPLE

Microsoft's own reference notes there is no distinct sample scenario beyond EARLIER's; the documented pattern is: `= EARLIEST(<column>)`, used identically to `EARLIER(<column>, <highest available level>)` whenever three or more calculated-column row-context loops are nested.

COMMON STUDENT MISTAKES

- Assuming EARLIEST is a "stronger" or generally preferred version of EARLIER — it is simply a shortcut for one specific nesting level; for single-level nesting, use EARLIER.
- Using EARLIEST when unsure how many levels of nesting exist — when in doubt, use EARLIER with an explicit numeric second argument so the intent is unambiguous.

6.13 LOOKUPVALUE

Returns: value

Category: Lookup function

Works without a relationship

Purpose: Returns a value from a column in another (or the same) table, for the row where one or more search conditions all match exactly — without requiring a modeled relationship between the tables. Ideal for "as-of" lookups (e.g. exchange rate on a specific date) that a simple relationship can't express.

SYNTAX

```
LOOKUPVALUE (
    <result_columnName>,
    <search_columnName>, <search_value>
    [, <search2_columnName>, <search2_value>]...
    [, <alternateResult>]
)
```

Parameter	Definition
result_columnName	Column that holds the value you want returned. Must be a plain column, not an expression.
search_columnName / search_value	One or more matching pairs — the column to search and the value it must equal.
alternateResult	Optional value returned when zero or more than one distinct match is found (otherwise BLANK or an error is returned).

STEP-BY-STEP USAGE

1. **First check whether a relationship already exists** between the two tables — if it does, use **RELATED** instead; it is more efficient and is what Microsoft's own documentation recommends over LOOKUPVALUE whenever a relationship is available.
2. If no relationship exists (or the match needs multiple conditions a relationship can't express, like matching on both currency *and* date), list every search column/value pair needed for a unique match.
3. Always supply an **alternateResult** in production formulas rather than relying on the default BLANK/error, so unmatched rows fail gracefully.
4. Test against a row you know should NOT find a unique match, to confirm the alternateResult behaves as expected.

WORKED EXAMPLE

```
Exchange Rate =
LOOKUPVALUE (
    'Currency Rate'[Average Rate],
    'Currency Rate'[CurrencyKey], [CurrencyKey],
    'Currency Rate'[DateKey], [OrderDateKey]
)

Sales Amount Local Currency = [Sales Amount] * [Exchange Rate]
```

Result: Each order looks up the exchange rate that was in effect for its specific currency *and* order date — a two-condition match that a single relationship column could not represent.

COMMON STUDENT MISTAKES

- Reaching for LOOKUPVALUE when a relationship already links the two tables — RELATED is faster and simpler for that case.
- Omitting `alternateResult` and being surprised by an error when a row has zero or multiple matches — always add a sensible fallback.
- Passing an expression as `result_columnName` — it must be a plain existing column reference.

6.14 SELECTEDVALUE

Returns: value

Category: Filter-context reader

Best practice replacement for HASONEVALUE + VALUES

Purpose: Returns the single value a column has been filtered down to, if there is exactly one; otherwise returns a fallback. It is the standard, readable way to answer "what did the user pick?" inside a card visual, tooltip, or title measure.

SYNTAX

```
SELECTEDVALUE ( <columnName> [, <alternateResult>] )
```

Parameter	Definition
<code>columnName</code>	The column to inspect. Must be a plain existing column, not an expression.
<code>alternateResult</code>	Optional; returned when zero or more than one distinct value remains in context. Defaults to BLANK().

STEP-BY-STEP USAGE

1. Identify the column whose "current single selection" you need to display or branch logic on.
2. Write `SELECTEDVALUE(Table[Column], "<fallback text>")`.
3. Place the measure in a card, a dynamic title, or as an input to further logic (e.g. IF branching on which product is selected).
4. Test with zero selections, one selection, and multiple selections to confirm the fallback text appears exactly when expected.

WORKED EXAMPLE

```
Selected Color = SELECTEDVALUE ( DimProduct[Color], "No Single Selection" )
```

Result: With one color filtered/selected (e.g. Black), the measure returns "Black"; at the report's grand-total row (all colors in context), it returns the fallback text "No Single Selection."

COMMON STUDENT MISTAKES

- Writing the older, more verbose equivalent
`IF(HASONEVALUE(Column), VALUES(Column), fallback)` out of habit — SELECTEDVALUE is the documented best-practice replacement and should be preferred in new formulas.
- Forgetting the default fallback is BLANK(), which can look like "nothing happened" in a card visual — always supply an explicit, meaningful alternateResult.
- Passing an expression instead of a plain column reference as the first argument.

7. Cross-Cutting Best Practices & Pitfalls

BEST PRACTICE

Prefer a plain Boolean filter argument (`Table[Column] = value`) over `FILTER(Table, Table[Column] = value)` when possible — the query engine can optimize Boolean filters far better than a full table scan wrapped in FILTER.

BEST PRACTICE

Prefer `REMOVEFILTERS` over bare `ALL()` when your only intent is "clear this filter" inside CALCULATE — it removes ambiguity about whether a table object was intended.

BEST PRACTICE

Prefer `SELECTEDVALUE` over the older `IF(HASONEVALUE(...), VALUES(...), ...)` pattern for readability and consistency.

CAUTION

Every filter function covered in this SOP is documented by Microsoft as **not supported in DirectQuery mode when used inside calculated columns or row-level security (RLS) rules**. If your model uses DirectQuery, test RLS and calculated-column logic that relies on these functions early — you may need to move the logic into a measure or an Import-mode table instead.

CAUTION

CALCULATE's overwrite-vs-add behavior (Section 6.7) is the single most common source of "why is my filter being ignored" bugs. When a measure's result doesn't change as expected after adding a CALCULATE filter, check whether an *inner* CALCULATE is silently overwriting an *outer* filter on the same column — and reach for KEEPFILTERS if you need an intersection instead.

FREQUENT MIX-UP

ALL vs. ALLSELECTED vs. ALLEXCEPT — students often use these interchangeably. Remember: ALL ignores every filter including slicers; ALLSELECTED keeps outer/report filters but drops the visual's own row/column grouping; ALLEXCEPT keeps only the columns you explicitly name and clears everything else on that table.

FREQUENT MIX-UP

CALCULATE vs. CALCULATETABLE — the choice is entirely determined by whether the first argument returns a single value (CALCULATE) or a table (CALCULATETABLE); mixing them up produces a clear DAX type-mismatch error that points directly at the fix.

8. Adjacent Functions Outside This SOP's Scope

These appear on the same Microsoft Learn "Filter functions" page but operate on a matrix's visual axis rather than the model's filter context, and apply only inside Power BI's Visual Calculations feature. Listed here so students recognize them and know where to look next.

Function	One-line description
FIRST / LAST / NEXT / PREVIOUS	Retrieve a value from the first, last, next, or previous row of a visual's axis.
INDEX / OFFSET / RANGE / WINDOW	Return one or more rows at a relative or absolute position within a visual's axis.
RANK / ROWNUMBER	Return the ranking or unique row number within an interval.
MOVINGAVERAGE / RUNNINGSUM	Return a moving average or running total along a visual's axis.
ORDERBY / PARTITIONBY / MATCHBY	Define sort order, partitions, and row-matching columns for window functions.
LOOKUP / LOOKUPWITHTOTALS	Visual-calculation-mode lookups against the current matrix, with or without inferred filters on totals.

9. Practice Exercises for Students

Work through these using the AdventureWorks DW sample model (or an equivalent star schema). Suggested approach: restate the goal, identify the filter context, pick a function from the Section 5 decision guide, write the DAX, and validate in a matrix — exactly as in the Section 4 procedure.

Exercise 1 — ALL vs ALLSELECTED

Build a matrix of Sales by Product Category (rows) with a Year slicer set to 2007 only. Create two measures: one using ALL(Sales) and one using ALLSELECTED(Sales) for the "% of total" column. Explain in one sentence why they return different numbers once the slicer is applied.

Exercise 2 — ALLEXCEPT

Write a measure that always shows total sales for the selected Country, ignoring City and State/Province filters, using ALLEXCEPT on your Geography table.

Exercise 3 — FILTER + CALCULATE

Write a measure for "High-Value Orders Total" that sums only orders where Sales Amount > 1000, using FILTER inside CALCULATE. Then rewrite it as a plain Boolean CALCULATE filter and confirm both give the same result — which one is the recommended pattern, and why?

Exercise 4 — KEEPFILTERS

Create two nested CALCULATE measures that both filter the same column with different values (similar to the Section 6.10 example). Show the result with and without KEEPFILTERS and explain the difference in your own words.

Exercise 5 — EARLIER

In a Product calculated column, rank products within their category by sales, highest first, using EARLIER inside FILTER/COUNTROWS. Then research and re-implement the same result using RANKX, and compare readability and performance.

Exercise 6 — SELECTEDVALUE + LOOKUPVALUE

Build a card visual that shows "Selected Region: <region>" using SELECTEDVALUE with a sensible fallback when multiple regions are selected. Separately, use LOOKUPVALUE to pull a region's target quota from a Targets table that has no relationship to your Sales table.

10. Quick Reference Summary Table

Function	Returns	Typical Container	One-line memory hook
FILTER	Table	Inside CALCULATE/CALCULATETABLE/ iterator	Row-by-row condition → subset of rows.
ALL	Table/ Column	Modifier or table expression	Clears filters (table, or named columns).
ALLEXCEPT	Table	Modifier	Clears everything except the named columns.
ALLSELECTED	Table/ Column	Modifier	Clears the visual's own grouping, keeps outer filters.
ALLNOBLANKROW	Table/ Column	Modifier	Like ALL, minus the relationship blank row.
ALLCROSSFILTERED	N/A	Modifier	Clears filters including many-to-many cross-filters.
CALCULATE	Scalar	Standalone measure body	Recompute a value under a new filter context.
CALCULATETABLE	Table	Standalone / feeds an iterator	CALCULATE for table results.
REMOVEFILTERS	N/A	Modifier	Explicit, unambiguous "clear this filter."
KEEPFILTERS	Wraps a filter	Inside CALCULATE/CALCULATETABLE	Intersect instead of overwrite.
EARLIER	Value	Inside nested row context (calc. column)	Reach one level out in a nested loop.
EARLIEST	Value	Inside nested row context (calc. column)	Reach the outermost level in a nested loop.
LOOKUPVALUE	Value	Standalone / calc. column	Relationship-free multi-condition lookup.
SELECTEDVALUE	Value	Standalone measure	"What's currently selected?" with a fallback.

11. References

Page	URL
Filter functions (DAX) — category page (primary reference for this SOP)	https://learn.microsoft.com/en-us/dax/filter-functions-dax
ALL	https://learn.microsoft.com/en-us/dax/all-function-dax
ALLCROSSFILTERED	https://learn.microsoft.com/en-us/dax/allcrossfiltered-function-dax
ALLEXCEPT	https://learn.microsoft.com/en-us/dax/allexcept-function-dax
ALLNOBLANKROW	https://learn.microsoft.com/en-us/dax/allnoblankrow-function-dax
ALLSELECTED	https://learn.microsoft.com/en-us/dax/allselected-function-dax
CALCULATE	https://learn.microsoft.com/en-us/dax/calculate-function-dax
CALCULATETABLE	https://learn.microsoft.com/en-us/dax/calculatetable-function-dax
EARLIER	https://learn.microsoft.com/en-us/dax/earlier-function-dax
EARLIEST	https://learn.microsoft.com/en-us/dax/earliest-function-dax
FILTER	https://learn.microsoft.com/en-us/dax/filter-function-dax
KEEPFILTERS	https://learn.microsoft.com/en-us/dax/keepfilters-function-dax
REMOVEFILTERS	https://learn.microsoft.com/en-us/dax/removefilters-function-dax
LOOKUPVALUE	https://learn.microsoft.com/en-us/dax/lookupvalue-function-dax
SELECTEDVALUE	https://learn.microsoft.com/en-us/dax/selectedvalue-function-dax
DAX sample model (AdventureWorks DW) used in Microsoft's official examples	https://aka.ms/dax-docs-samples

All syntax, parameter definitions, and worked examples in this SOP are adapted directly from the Microsoft Learn pages listed above (accessed 30 August 2026). Numeric results shown in examples are Microsoft's own published sample outputs against the AdventureWorks DW model and will differ if you practice against a different dataset — use them to validate your understanding of the *mechanism*, not as universal constants.